批处理调度与吞吐延迟权衡

摘要：本文讨论在资源有限的单机上调度批任务与交互请求的取舍。核心结论是：批处理提升吞吐的收益存在拐点，越过拐点之后继续增大批量只会增加延迟；而在单用户场景里，延迟才是用户唯一能感知的指标。

1 引言。batching trades latency for throughput and the knee of the curve is where it stops helping。这条曲线的前半段很诱人：批量翻倍、吞吐几乎翻倍。但拐点之后每增加一点批量都要用显著更多的延迟去换，因此在设备已接近满载时继续加批量是没有意义的。

2 目标函数。在本地单用户场景下吞吐不是目标延迟才是。吞吐指标在多用户服务里代表成本效率，而在本地单机场景里它只影响后台任务完成得多快。把吞吐当作优化目标会得到“后台任务更快、但每次提问都更慢”的结果，与用户诉求相反。

3 队列设计。队列深度必须设上限否则延迟会随突发流量线性恶化。无上限队列在过载时不会立刻崩溃，它会先把延迟推到不可接受，再在某一刻整体超时。这种失败方式比立即拒绝更难排查，因为日志里只有超时，看不到过载的起点。

4 优先级。优先级反转会让交互请求排在批任务之后。典型成因是批任务先入队且批量很大，而交互请求必须等它整批执行完才能轮到。可行做法是为交互路径预留执行槽，或给批任务设置可抢占的时间片。

5 重试策略。退避重试必须带抖动否则所有客户端会同时重试。抖动的作用是把重试时刻散开，否则失败的请求会在同一个时间点再次同时到达，把一次短暂的过载放大成持续的拥塞。

表 1 调度参数与影响。批量大小 影响吞吐与单次延迟 ；队列上限 影响过载行为 ；优先级槽位 影响交互延迟上界 ；重试抖动 影响拥塞恢复速度 。

6 背压。当上游产生速度持续高于处理速度时，拒绝是唯一能保持系统可用的选择。具体地，过载时应当拒绝而不是排队因为排队的请求最终也会超时，因此让请求在队列里等待只是把失败推迟到更晚，并且让用户多等了一段时间才看到同样的失败结果。

7 超时预算。超时值应当按调用链累加而不是每层各设一个。若内层超时大于外层，外层会先超时并丢弃内层的结果，造成“任务实际成功但上游认为失败”的重复执行。

8 观测。除平均延迟外必须记录分位数，因为调度问题几乎都体现在尾部。平均延迟对少量极慢请求极不敏感，而用户感知到的恰恰是那些极慢的请求。

9 与批处理的边界。后台任务可以接受秒级延迟，交互请求不能。因此调度的第一原则是按路径分类，而不是按任务大小分类；把大小当作分类依据会让一个很小的交互请求因为排在批任务之后而变慢。

10 结论。调度问题的实质是分配稀缺资源，而不是优化单个指标。先把交互路径的延迟上界固定下来，再让批任务使用剩余容量，这样得到的系统在过载时表现为变慢而不是不可用。

11 时间片与抢占。批任务如果要能被打断，就必须以可恢复的粒度执行：每处理完一个小批次就写一次进度，被抢占后从上次进度继续。若批任务内部不可中断，那么“预留槽位”只是把延迟从一分钟变成一分钟零几秒，并不能真正保证交互请求的响应时间。

12 饥饿与公平性。为交互请求预留槽位之后，批任务在持续高负载下可能长期得不到执行。因此预留比例不能太大，并且应当记录批任务的排队时长，当它超过阈值时给出告警，而不是等到用户发现后台任务一直没有进展。

13 与缓存的关系。调度层不感知缓存，但缓存会显著改变调度效果：命中缓存的请求耗时极短，如果它们与未命中的请求排在同一个队列里，队列深度就无法反映真实负载，按队列深度触发的背压也会滞后。

14 容量估算。容量必须按分位数而不是平均值估算：若按平均延迟估算，系统在容量利用率七成

时就已经有相当比例的请求超时。实践上按第九十五百分位留出余量，可以让过载出现在明确的点上而不是随时可能发生。

１６　时间窗与突发。请求到达很少均匀，上传场景尤其如此：用户往往一口气选十几篇文档。因此容量规划必须按突发而不是按平均到达率来做，否则在突发到来时队列瞬间打满，之前按均值算出的余量毫无用处。

１７　令牌桶与漏桶。令牌桶允许短时突发、长期受限，适合交互路径；漏桶输出恒定，适合后台批任务。把两者的位置放反会得到“交互请求被平滑掉、批任务却能突发”的结果，与初衷相反。

１８　背压的实现层次。背压可以在入口拒绝、可以在队列上限处丢弃、也可以在上游放慢生产。第三种体验最好但需要上游配合，在单机部署里通常只能做前两种，因此拒绝策略必须显式设计并给出可读的提示。

１９　优先级与公平。纯优先级会让低优先级任务长期得不到执行；纯轮转又会牺牲交互延迟。常见折中是加权轮转：按权重分配执行机会，同时为交互路径保留一个从不被占用的槽位。

２０　观测指标。调度层的核心指标是排队时长与丢弃率，而不是吞吐。吞吐高但排队时长长的系统，用户体验依然很差；而丢弃率上升说明容量已经不足，需要在扩容之前先确认是否真的应该接这么多任务。

２１　与检索的关系。批量任务完成后通常要重建索引，而重建期间检索会变慢甚至短暂不可用。因此重建应当安排在低峰，或者采用增量更新把一次全量重建拆成多次小更新。

２２　单机与集群的差别。集群可以靠扩副本消化突发，单机只能靠队列与拒绝。因此从集群迁移到单机时，原本被掩盖的排队问题会立刻暴露，这类问题的根因往往不是算法而是容量假设失效。

２３　超时与取消。排队中的请求应当允许取消，并立刻从队列移除；否则用户点了取消之后还要等到它被执行再被丢弃，这期间它占着队列位置并挤压其他请求。

２４　幂等与去重。同一任务被重复提交时应当合并而不是并行执行，否则重复请求会各自占用容量，把可用容量消耗在完全相同的计算上。

２５　资源隔离。把编码等重资源阶段单独排队，而不是与轻量阶段共用一个队列，可以避免轻量请求被重任务堵住。隔离的粒度应当按资源类型而不是按业务类型划分。

２６　容量的保守估计。容量必须按最坏输入而不是典型输入估算：一篇超长文档的编码时间可能是普通文档的十倍，按典型值规划会让系统在遇到长文档时突然拥塞。

２７　降级策略。过载时除了拒绝，还可以降级：例如把候选数从二十降到十、或跳过重排直接返回融合结果。降级必须显式可见，否则用户会把降级后的质量当成系统本来的水平。

２８　恢复过程。过载结束后队列里的积压会集中执行，形成第二次拥塞。因此恢复阶段应当限速，否则系统会在“过载—积压—再过载”之间反复震荡。

３０　调度的可观测性。调度决策本身也需要记录：某个请求为什么被拒、为什么排在后面、按什么权重被选中。缺少这些记录时，用户报告的“有时候快有时候慢”无法被归因，只能靠猜。

３１　与调度无关的另一种慢。检索缓存未命中、模型冷启动、以及磁盘上的索引首次加载，都会表现为偶发的慢请求。把它们误判为调度问题会导致在错误的层次上优化，因此慢请求必须按原因分类统计。

３２　配置的可调性。队列上限、并发数、超时阈值都应当可运行时调整，否则每次调参都要重启，而重启本身会清空队列、掩盖问题现场。

３３　排队与排序的语义。队列的排序键决定了公平性：按到达时间排是先进先出，按预计耗时排会优先短任务，按优先级加权排则取决于权重设计。三种排序键在过载时的行为差别极大，因此必须显式选定而不是让实现细节决定。

３４　短任务优先的代价。优先短任务能显著降低平均等待时间，但长任务可能被无限推迟。若长任务恰好是用户主动触发的，体验会变得极差；因此这一策略只适合后台的批量补算任务。

３５　饥饿检测。无论采用哪种策略，都应当检测是否存在长期未被执行的请求，并在超过阈值时临时提升其优先级。缺少这一层保护时，饥饿问题只会在极端负载下暴露，而那时已经很难复现。

３６　与前端的分工。前端不应自行重试失败请求，否则它会绕过调度层的退避与去重，把一次拥塞放大成多轮重试风暴。重试应当由调度层统一负责，前端只负责展示状态。

３７　容量与质量。过载时降级会改变输出质量，因此降级必须与评测口径联动：被降级的请求应当被标记，在统计质量指标时单独归类，否则降级期的指标会污染长期基线。

３８　结论补充。把批任务与交互请求分成两条独立的容量通道，是本地部署里最简单也最有效的调度手段。它不需要复杂的优先级算法，只需要在入口处按路径分流，并各自设置独立的队列上限。